
Programmation orienté objet en C++

Cours

Enseignant :
M. Eric DARRIGAND

Étudiant :
Nathan HASCOET

10 septembre 2026

Table des matières

1	Introduction au C++	1
1.1	Langage compilé	1
1.2	Entrée/Sorties standard	1
1.3	Notion de classe en C++	2
1.4	Structuration d'un code en fichiers	2
1.5	Premières lignes d'un .cpp	2
1.6	Exemple de classe	3
1.7	Arguments avec valeur par défaut dans une fonction	4
1.8	Surcharge de fonctions	5
1.9	Transmission par valeurs, par pointeurs ou par références	5

1 Introduction au C++

1.1 Langage compilé

```
g++ -C compilation
    -std=c++11
    -Wall
    -fsanitize=address
```

Organisation du code

- type et variables.
- fonctions.
- classes.
- espaces de noms (*namespace*).

(std : :)sqrt

↑

namespace de la lib standard (STL).

La STL contient : `#include <cmath>`
`<cstdlib>`
using std : :sqrt;

1.2 Entrée/Sorties standard

En C++, on passe par les flots : "*cin*", "*cout*", ">>" et "<<".

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int num_annee;
5      cout << "Donner le num de l annee" << endl;
6      cin >> num_annee;
7      cout<<"Bonjour et bonne rentreee" << num_annee << endl;
8  }
9
```

Listing 1 – Exemple de code en C++

1.3 Notion de classe en C++

Définition de base d'une classe :

```
1  class MaClasse{
2      private :
3          // donnees membres
4          int donnee 1;
5          int donnee 2;
6      public :
7          //constructeurs (optionnels).
8          MaClasse(...);
9          // fonctions membres ou methodes
10         int fonc1();
11         double fonc2(int);
12     }
13
```

Listing 2 – Définition d'une classe

Les fonctions membres peuvent être définies ailleurs.

```
1  MaClasse::MaClasse(...){...}
2  int MaClasse::fonc1(){...}
3  double MaClasse::fonc2(int i){...}
4
```

Listing 3 – Définition d'une classe

Usage : dans un programme appelant.

```
MaClasse obj1;          cout <<obj1
MaClasse* obj1_p;       cout <<obj
```

1.4 Structuration d'un code en fichiers

→ Des fichiers .hpp (ou .h++ ou .h)

Ce sont des headers (essentiellement des déclarations)

↔ définitions de classes

```
1      #ifndef MACLASSE_H
2      #define MACLASSE_H
3      class MaClass{
4          ...
5      }
6      #endif //MACLASSE_H
7
```

Listing 4 – Exemple de header

→ Des fichiers .cpp (en .c++)

↔ programmation des fonctions déclarées et non définies dans les hpp.

1.5 Premières lignes d'un .cpp

```

1  #include "MesClasses.hpp"
2  #include <iostream>
3  #include <cmath>
4  using std::cin;
5  using std::sqrt;
6

```

Listing 5 – Début d'un .cpp

1.6 Exemple de classe

La classe complexe.

"Complexe".

→ 2 données membres

partie réelle (double)

partie imaginaire (double)

→ construction avec 2 arguments d'entrées de type double pour définir les données membres

```

1  #ifndef COMPLEXE_H
2  #define COMPLEXE_H
3  class Complexe{
4  private:
5      double re, im;
6  public :
7      Complexe(double a, double b);
8  };
9  #endif // COMPLEXE_H
10

```

Listing 6 – complexe.hpp

```

1  #include "complexe.hpp"
2
3  Complexe::Complexe(double a, double b){
4      re = a;
5      im = b;
6  }
7

```

Listing 7 – complexe.cpp

Ces exemples et d'autre se trouve dans le livre que l'on peut récupérer à l'adresse suivante :

<https://www.stroustrup.com/>

Des classe à metre de préférence dans le .hpp :

- ★ Définitions de types, classes, déclarations de classes.
- ★ Déclarations de fonctions.
- ★ Déclarations de variables globales
- ★ Définitions de variables consantes.

★ Définitions des fonctions "inline" (à réserver aux fonctions courtes).

```
1  #ifndef COMPLEXE_H
2  #define COMPLEXE_H
3  class Complexe{
4  private:
5  double re, im;
6  public :
7  Complexe(double a, double b){
8      re = a;
9      im = b;
10 } // fonction "inline"
11 };
12 #endif // COMPLEXE_H
13
```

Listing 8 – Exemple de fonction "inline" (complexe.hpp)

Le pré-processeur va remplacer à chaque occurrence la fonction Complexe par le script qui est donné. Ce qui peut rendre le code très lourd à exécuter.

Dans une .hpp, on évitera :

★ Définition d'une variable non constante.

★ Usage de la directive "using" (interdit : *stroustrup* [2014] p.424).

```
1  #include "complexe.hpp"
2  int main(){
3      Complexe z1(1.5,2.5);
4      Complexe z2;
5  }
6
```

Listing 9 – Exemple d'un main.cpp

1.7 Arguments avec valeur par défaut dans une fonction

```
1  int sum(int a, int b = 1){return a+b;}
2  int a=2,b=4,c,d,e;
3  c = som(a); // c=3
4  d = sum(b); // d=5
5  e = sum(a,b); // e=6
6
```

Listing 10 – Exemple 1

```
1  int sum(int a=0, int b = 1){return a+b;}
2  int a=2,b=4,c,d,e,f;
3  c = som(a); // c=3
4  d = sum(b); // d=5
5  e = sum(a,b); // e=6
6  f = sum(); // f=1
7
```

Listing 11 – Exemple 2

1.8 Surcharge de fonctions

```
1  int add(int a, int b){return a+b};
2  double add(double a, double b){return a+b};
3  int a=0,b=1,e;
4  double c=0.5,d=1.5,f,g;
5  e = add(a,b); // e=1
6  f = add(c,d); // f=2
7  g = add(c,b); // crash : refus par le compilateur (ambigu)
```

Listing 12 – Exemple de surcharge de fonctions

1.9 Transmission par valeurs, par pointeurs ou par références

```
1  int a=2;
2  int* a_p = &a;
3  carre(a_p);
```

Listing 13 – Transmission par pointeur

Une référence est un "alias" vers une variable déjà déclaré :

- L'accès à la référence est semblable à l'accès à la variable elle-même.
- Une référence réfère toujours à l'objet auquel elle a été initialisée.
- Une référence est nécessairement rattachée à un objet.

```
1  int var=1;
2  int& r(var); //r est une reference sur la variable "var".
3  int x=r; // x prend la valeur 1.
4  r=2; // la valeur de var devient 2 mais x ne change pas.
5  r++; // la valeur de var devient 3.
6  int* p = &r; // p pointe sur la variable var.
```

Listing 14 – Exemple d'une référence

```
1  int add(int a, int b){return a+b};
2  int add(int* a_p, int* b_p){return *a_p+*b_p};
3  int add(int& a, int& b){return a+b};
4  int carre(int& a, int& b){return a*=a};
```

Listing 15 – Transmission par référence

La première et la troisième fonction ne peuvent pas cohabiter dans le même script, car le programme ne saura pas dire lequel utiliser puisque la référence peut-être utilisée par les deux fonctions.